**Universidad Cooperativa de Colombia**

[Programa o facultad]

**Desarrollo Sostenible**

Actividad evaluativa – Momento 2

# CeroHambre: plataforma web para recuperar alimentos y contribuir al ODS 2 – Hambre Cero

**Informe técnico del proyecto · ODS 2 – Hambre Cero**

**Integrantes**

Jesus Quintero

[Nombre del integrante 2]

[Nombre del integrante 3]

**Docente**

[Nombre del docente]

Colombia, octubre de 2026

# Tabla de contenido


**1. Enlaces de acceso y credenciales de prueba ..... 3**

**2. Descripción de la solución y contribución al ODS 2 ..... 4**

2.1 Problema que se atiende ..... 4
2.2 Solución propuesta ..... 4
2.3 Contribución al ODS 2 – Hambre Cero ..... 4
2.4 Usuarios y roles ..... 5

**3. Funcionamiento y arquitectura ..... 6**

3.1 Arquitectura de tres capas ..... 6
3.2 Flujo principal ..... 6
3.3 Estados y reglas de negocio ..... 7
3.4 Estructura del repositorio ..... 7
3.5 Principales servicios de la API ..... 8

**4. Capturas de pantalla de las vistas principales ..... 9**

4.1 Página de inicio ..... 9
4.2 Registro e inicio de sesión ..... 10
4.3 Panel principal ..... 11
4.4 Donaciones disponibles ..... 12
4.5 Publicación de una donación ..... 13
4.6 Solicitudes ..... 14
4.7 Entregas ..... 15
4.8 Reportes ..... 16

**5. Base de datos ..... 17**

5.1 Motor de base de datos ..... 17
5.2 Modelo relacional ..... 17
5.3 Fragmento del esquema ..... 18
5.4 Diagrama entidad-relación ..... 19
5.5 Cardinalidades ..... 19
5.6 Normalización (3FN) ..... 19
5.7 Integridad de los datos ..... 20
5.8 Archivos entregados en /database ..... 20

**6. Tecnologías y justificación de los cambios ..... 21**

6.1 Tecnologías utilizadas ..... 21
6.2 Cambios respecto al Momento 1 ..... 21
6.3 Despliegue en Render ..... 22

**7. Validaciones, seguridad y pruebas ..... 23**

7.1 Validaciones y manejo de errores ..... 23
7.2 Seguridad ..... 23
7.3 Pruebas realizadas ..... 23

**8. Conclusiones ..... 24**

## 1. Enlaces de acceso y credenciales de prueba

Este informe acompaña la entrega del Momento 2. A continuación se relacionan el código fuente, la aplicación publicada y las credenciales para probar cada rol del sistema.

| Elemento | Información |
|---|---|
| Repositorio en GitHub | [https://github.com/USUARIO/REPOSITORIO] |
| Acceso del docente al repositorio | [Repositorio público, o usuario de GitHub del docente invitado como colaborador] |
| Aplicación desplegada (Render) | [https://NOMBRE-DE-LA-APP.onrender.com] |
| Base de datos en el repositorio | Carpeta `/database`: `schema.sql`, `seed.sql`, `cerohambre_backup.sql` y `DER.png` |

### Usuarios de prueba

| Rol | Correo | Contraseña | Qué se puede probar |
|---|---|---|---|
| Donante | `donante@cerohambre.com` | `Donante123*` | Publicar donaciones, aceptar o rechazar solicitudes, registrar y actualizar entregas. |
| Organización | `organizacion@cerohambre.com` | `Organizacion123*` | Consultar alimentos disponibles, solicitar cantidades y seguir sus solicitudes y entregas. |

Las secciones *Inicio* y *Reportes* son públicas; el resto de las vistas exige iniciar sesión. También es posible crear cuentas nuevas desde la vista de registro.

> **Nota sobre el servicio gratuito de Render:** si la aplicación lleva unos minutos sin recibir visitas, la primera carga puede tardar cerca de un minuto mientras el servicio se reactiva.

## 2. Descripción de la solución y contribución al ODS 2

### 2.1 Problema que se atiende

Restaurantes, panaderías, supermercados y productores generan con frecuencia excedentes de alimentos en buen estado que terminan desperdiciados, mientras que fundaciones, comedores comunitarios y hogares infantiles necesitan esos alimentos y no cuentan con un canal ágil para saber qué hay disponible, en qué cantidad, dónde recogerlo y hasta cuándo es aprovechable. La falta de información oportuna y de coordinación hace que el alimento se venza antes de llegar a quien lo necesita.

### 2.2 Solución propuesta

**CeroHambre** es una aplicación web que conecta a los **donantes** de alimentos con las **organizaciones** que atienden a población vulnerable. El donante publica los alimentos disponibles (tipo, cantidad, unidad, fecha de vencimiento y lugar de recogida); la organización consulta la oferta y solicita la cantidad que necesita; el donante acepta o rechaza la solicitud y registra la entrega; y el sistema deja trazabilidad de cada paso y la resume en reportes de impacto.

La aplicación controla las reglas que evitan errores en la operación: no se puede solicitar más de lo disponible, no se pueden pedir alimentos vencidos o ya reservados y no se pueden duplicar solicitudes activas sobre la misma donación.

### 2.3 Contribución al ODS 2 – Hambre Cero

El Objetivo de Desarrollo Sostenible 2 busca poner fin al hambre, lograr la seguridad alimentaria y mejorar la nutrición. CeroHambre contribuye de manera directa a la **meta 2.1** (acceso de las personas en situación de vulnerabilidad a una alimentación suficiente y nutritiva) y de forma complementaria a la **meta 12.3** (reducir el desperdicio y las pérdidas de alimentos). Lo hace de cuatro maneras:

- **Acceso:** los alimentos excedentes llegan a organizaciones que los distribuyen a personas que los necesitan.
- **Reducción del desperdicio:** la fecha de vencimiento y el estado de cada donación permiten priorizar y aprovechar los alimentos antes de que se pierdan.
- **Coordinación y trazabilidad:** cada alimento pasa por estados verificables (disponible, reservado, entregado) y cada entrega por su propio seguimiento.

- **Medición:** la vista de reportes calcula los alimentos recuperados, las entregas realizadas y las organizaciones atendidas a partir de los registros del sistema.

CeroHambre es una herramienta de articulación: no reemplaza la logística de las organizaciones ni mide por sí sola cuántas personas son atendidas. Las cifras de los reportes dependen de los datos que registran los usuarios; los valores que aparecen en las capturas de este informe corresponden a los datos de ejemplo incluidos en el proyecto.

### 2.4 Usuarios y roles

| Rol | Quién es | Qué puede hacer |
|---|---|---|
| Donante | Restaurantes, panaderías, supermercados, productores. | Publicar, editar y eliminar sus donaciones; aceptar o rechazar solicitudes; registrar la entrega y cambiar su estado (pendiente, en camino, entregada, cancelada). |
| Organización | Fundaciones, comedores comunitarios, hogares infantiles. | Consultar alimentos disponibles; solicitar una cantidad; ver el estado de sus solicitudes y entregas. |
| Visitante | Cualquier persona. | Ver la página de inicio, las donaciones disponibles y los reportes públicos. |

# 3. Funcionamiento y arquitectura

## 3.1 Arquitectura de tres capas




El mismo servidor Express entrega los archivos del frontend y la API (`/api/...`), de modo que la aplicación completa se publica como un único servicio web en Render y se conecta a una base de datos PostgreSQL administrada. Las credenciales se leen de variables de entorno.

## 3.2 Flujo principal

1. El donante publica la donación
   estado: disponible
2. La organización la solicita
   solicitud: pendiente
3. El donante acepta
   solicitud: aceptada · donación: reservado
4. Se registra y envía la entrega
   entrega: pendiente → en camino
5. Se confirma la entrega
   entrega: entregada · solicitud: completada · donación: entregado

## 3.3 Estados y reglas de negocio

| Entidad | Estados | Reglas principales |
|---|---|---|
| Donación | disponible, reservado, entregado | Una donación disponible cuya fecha de vencimiento ya pasó se muestra como vencida y no puede solicitarse. |
| Solicitud | pendiente, aceptada, rechazada, completada | La cantidad pedida no puede superar la disponible. Al aceptar una solicitud, la donación queda reservada y las demás solicitudes pendientes se rechazan. |
| Entrega | pendiente, en camino, entregada, cancelada | Solo existe una entrega por solicitud. Al entregar, la solicitud se completa y la donación pasa a entregada; si se cancela, la donación vuelve a estar disponible. |

## 3.4 Estructura del repositorio

```
Backend/        API REST: Server.js, Config/, Routes/, controllers/,
                middleware/, utils/, scripts/ y tests/
Frontend/       Interfaz: páginas HTML, Css/Styles.css y Js/App.js
database/       schema.sql, seed.sql, cerohambre_backup.sql, DER.png/svg
docs/           Capturas del sistema e informe
.env.example    Variables de entorno necesarias (sin valores reales)
render.yaml     Despliegue automático en Render (Blueprint)
README.md       Instalación, ejecución, despliegue y API
```

### 3.5 Principales servicios de la API

| Método y ruta | Función | Acceso |
|---|---|---|
| POST /api/usuarios | Registrar usuario | Público |
| POST /api/usuarios/login | Iniciar sesión (entrega el token) | Público |
| GET /api/donaciones | Listar donaciones | Público |
| POST /api/donaciones | Publicar una donación | Donante |
| POST /api/solicitudes | Solicitar una donación | Organización |
| PUT /api/solicitudes/:id | Aceptar o rechazar una solicitud | Donante |
| POST /api/entregas · PUT /api/entregas/:id | Registrar y actualizar una entrega | Donante |
| GET /api/reportes | Indicadores de impacto | Público |
| GET /api/salud | Estado del servidor y de la base de datos | Público |

## 4. Capturas de pantalla de las vistas principales

Las capturas muestran la aplicación en funcionamiento con los datos de ejemplo del proyecto (`database/seed.sql`). Se recorre el flujo completo: registro e ingreso, publicación de una donación, solicitud por parte de una organización, aceptación, entrega y reportes.

### 4.1 Página de inicio

La página de inicio presenta el propósito del proyecto, explica el funcionamiento en cuatro pasos y muestra las cifras de impacto leídas de la base de datos. Desde allí se accede al registro y al inicio de sesión.

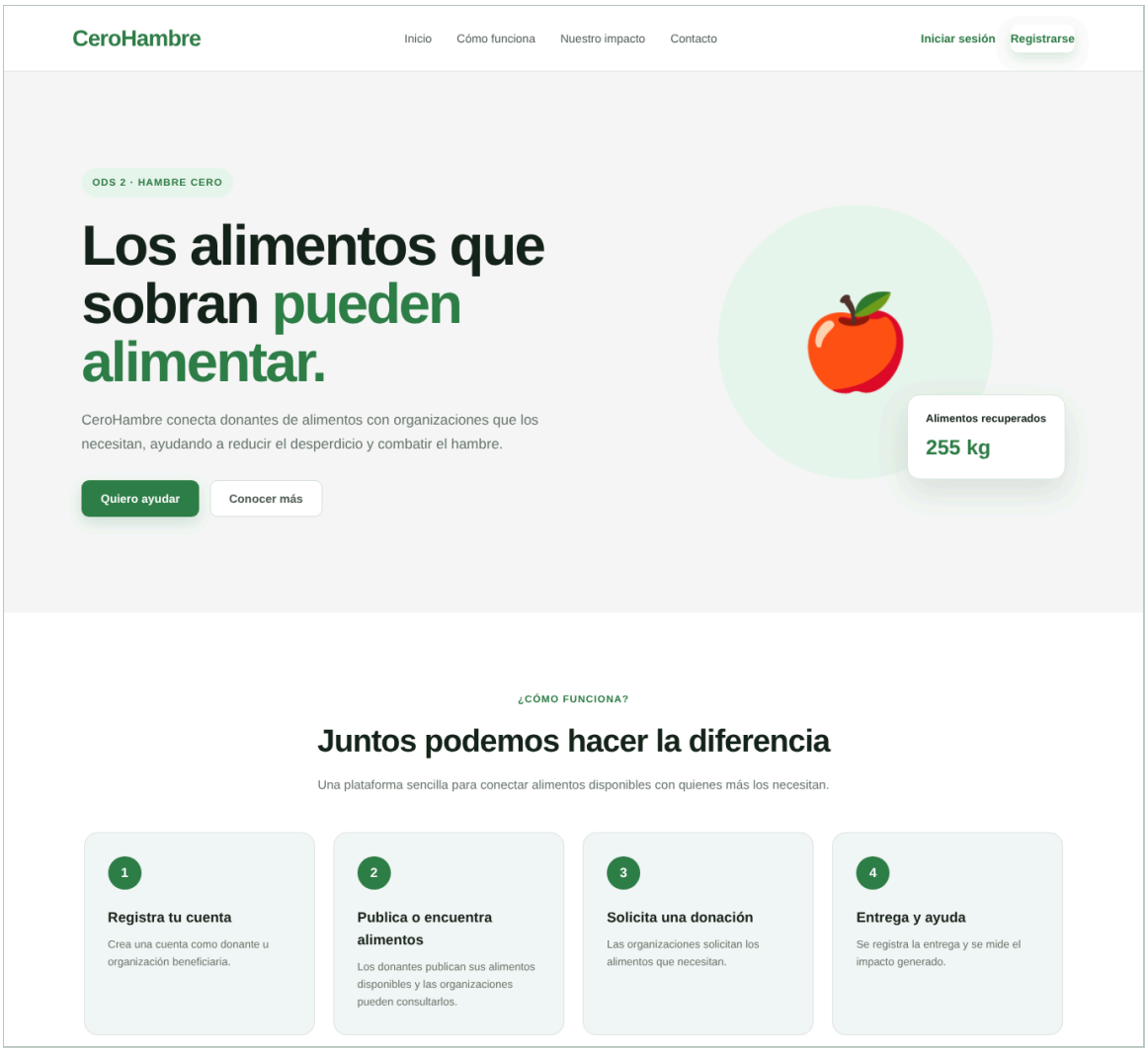



**Figura 1.** Página de inicio de CeroHambre.

## 4.2 Registro e inicio de sesión

En el registro cada usuario elige su tipo (donante u organización), indica su ubicación y define una contraseña de al menos 8 caracteres. El inicio de sesión valida las credenciales en el servidor y muestra un mensaje cuando son incorrectas.

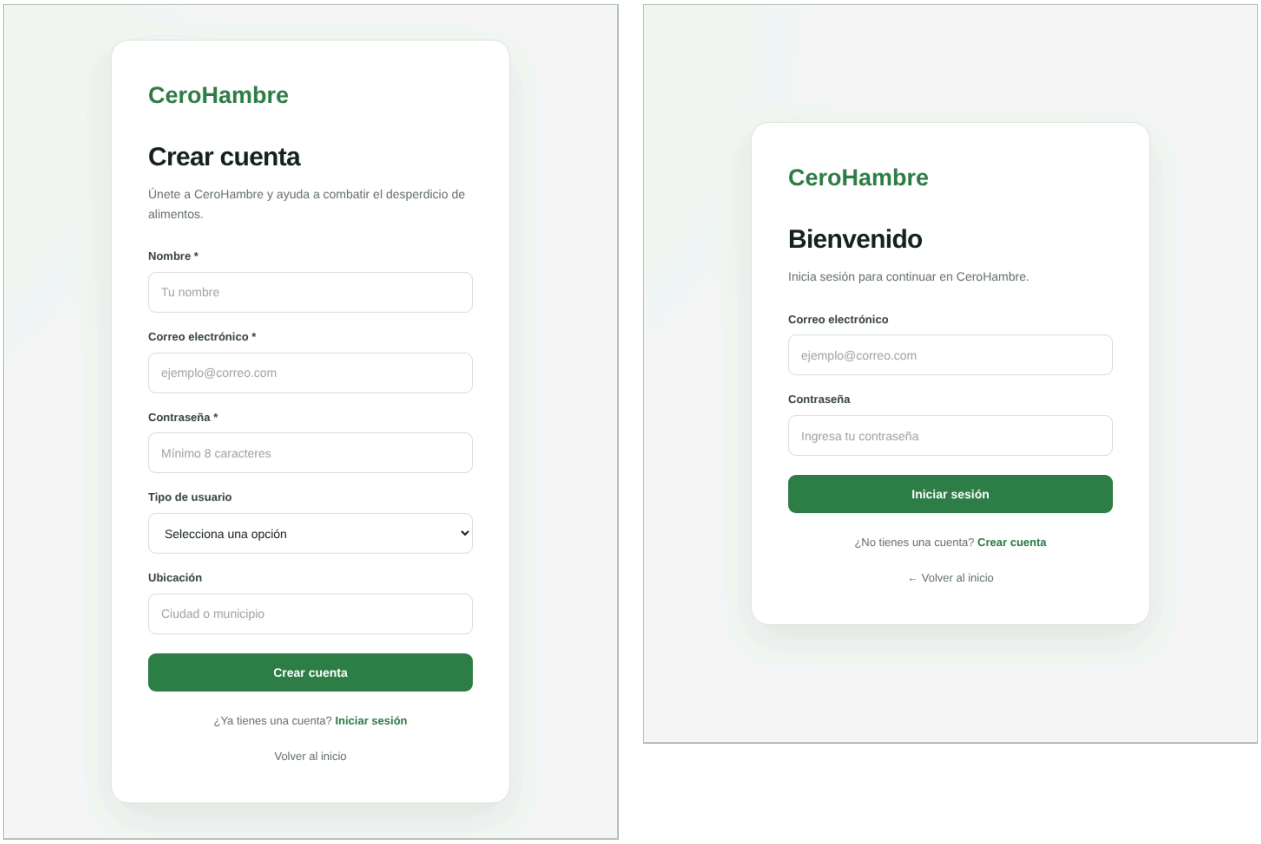



**Figura 2.** Registro de usuarios (izquierda) e inicio de sesión (derecha).

## 4.3 Panel principal

Tras iniciar sesión, el usuario llega a su panel: contadores de donaciones, solicitudes, entregas y alimentos recuperados, accesos rápidos a las funciones principales y actividad reciente. El menú y los botones se adaptan al rol: solo el donante ve las opciones de publicar donaciones y gestionar entregas.

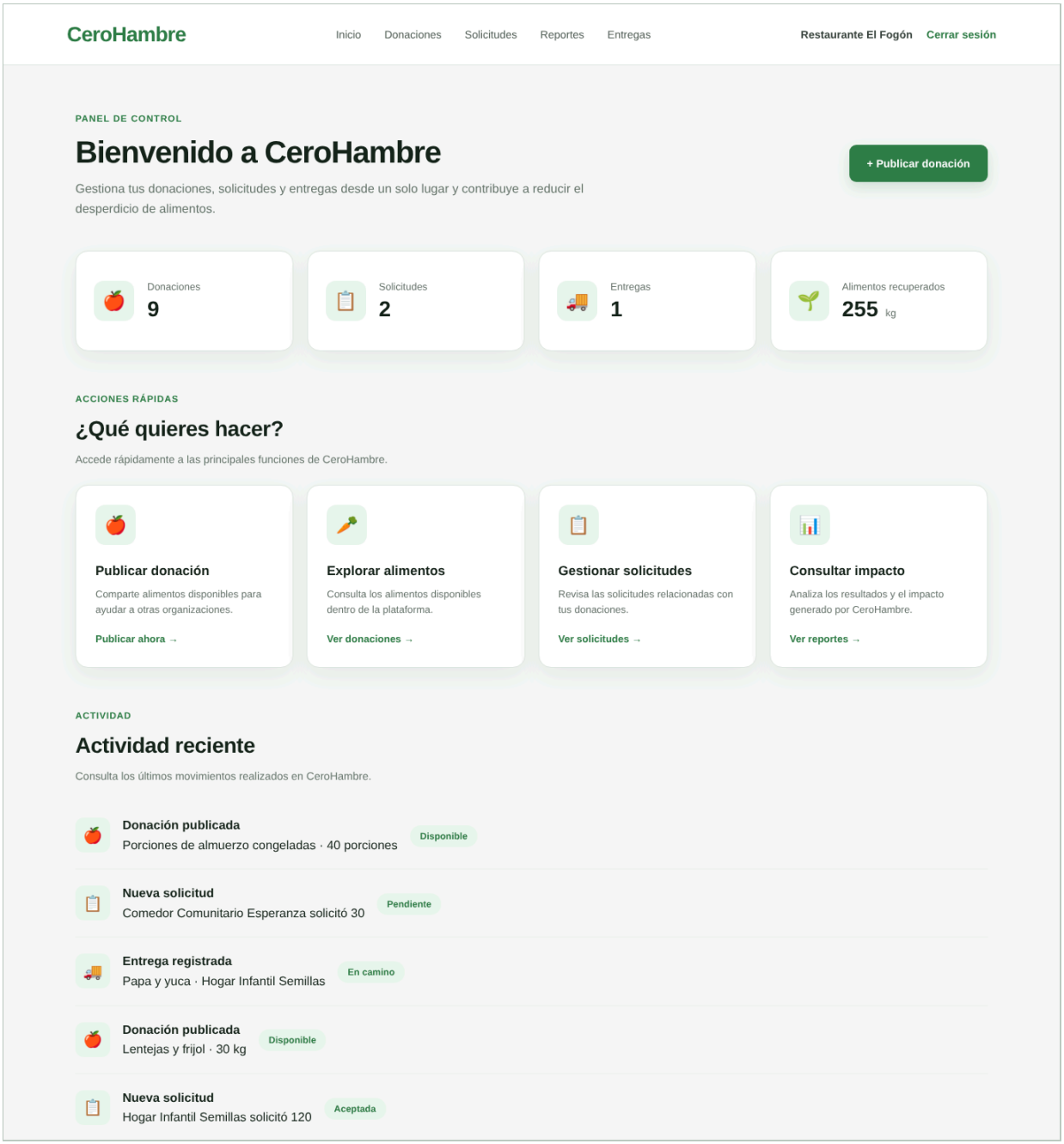



**Figura 3.** Panel principal de un usuario donante.

## 4.4 Donaciones disponibles

La vista de donaciones lista los alimentos con su cantidad, ubicación, fecha de vencimiento, donante y estado (disponible, reservado o entregado). Las organizaciones pueden solicitar desde cada tarjeta.

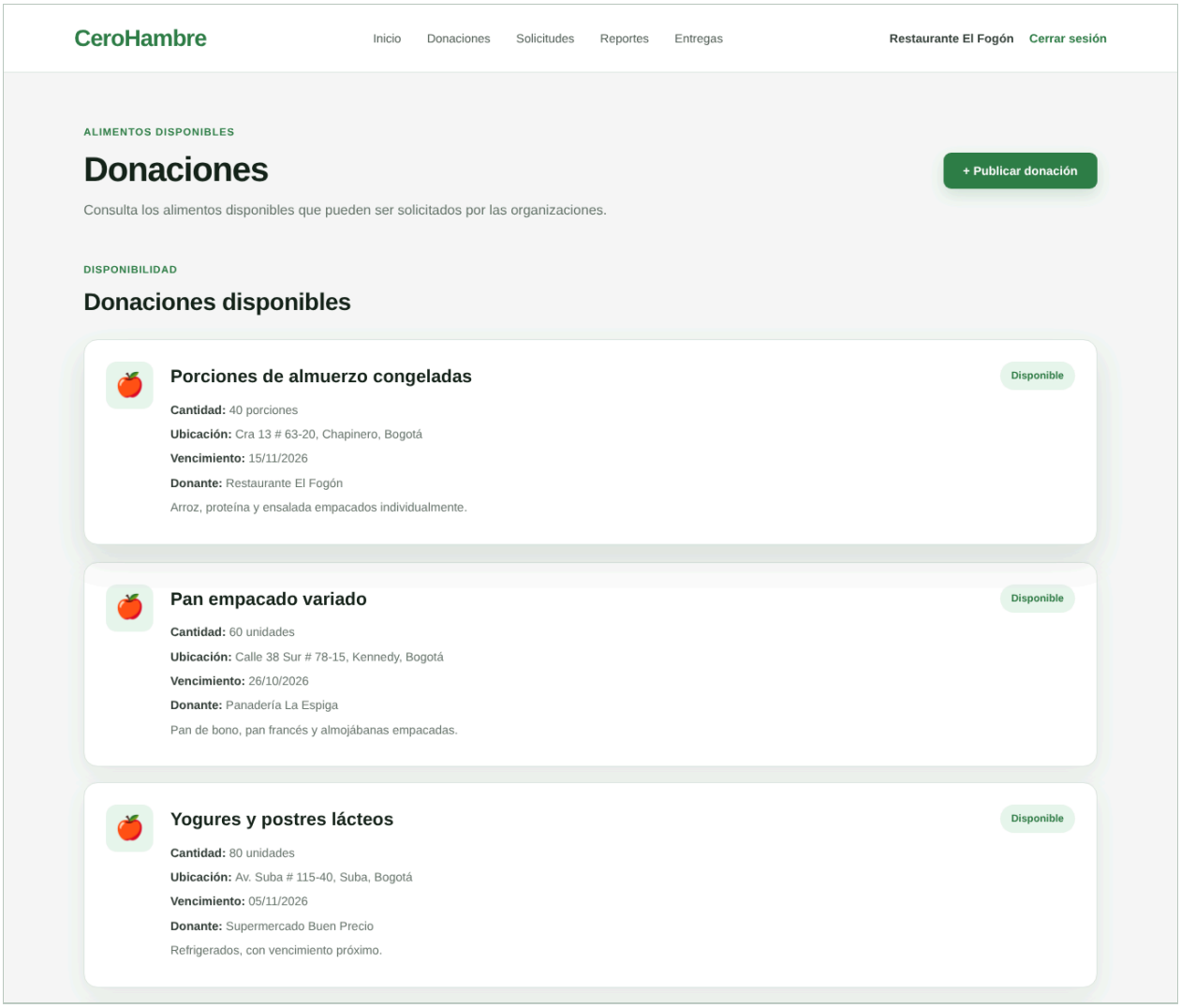



**Figura 4.** Listado de donaciones.

## 4.5 Publicación de una donación

El donante publica nuevas donaciones con un formulario que valida los campos obligatorios, que la cantidad sea positiva y que la fecha de vencimiento no sea pasada; el servidor repite estas validaciones.

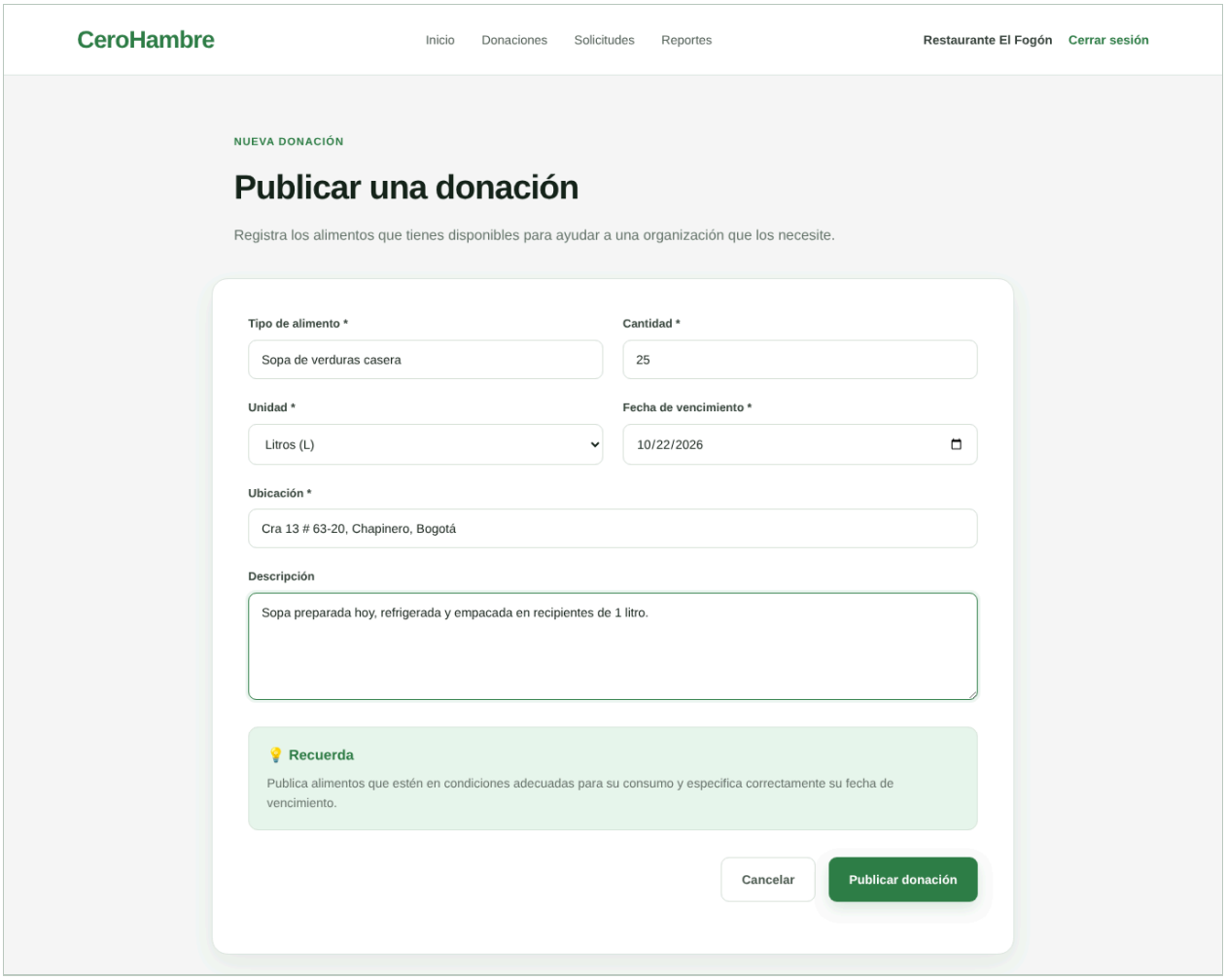



**Figura 5.** Formulario para publicar una donación.

## 4.6 Solicitudes

La organización solicita una cantidad desde la tarjeta de la donación; el sistema valida que no supere lo disponible y muestra el error en la misma ventana. El donante ve las solicitudes recibidas y puede aceptarlas o rechazarlas; al aceptar, la donación queda reservada.

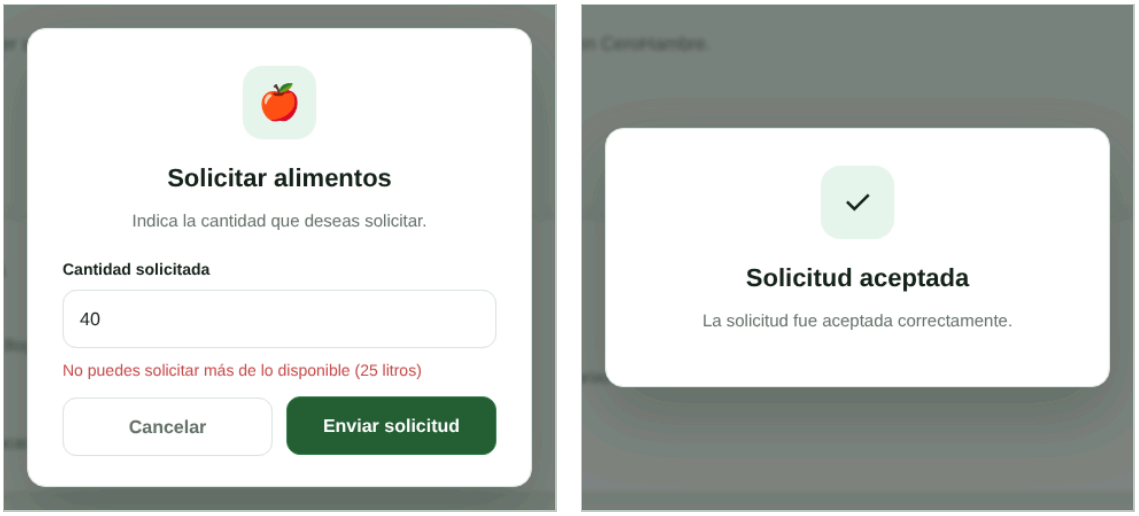



**Figura 6.** Validación de la cantidad solicitada (izquierda) y confirmación al aceptar una solicitud (derecha).

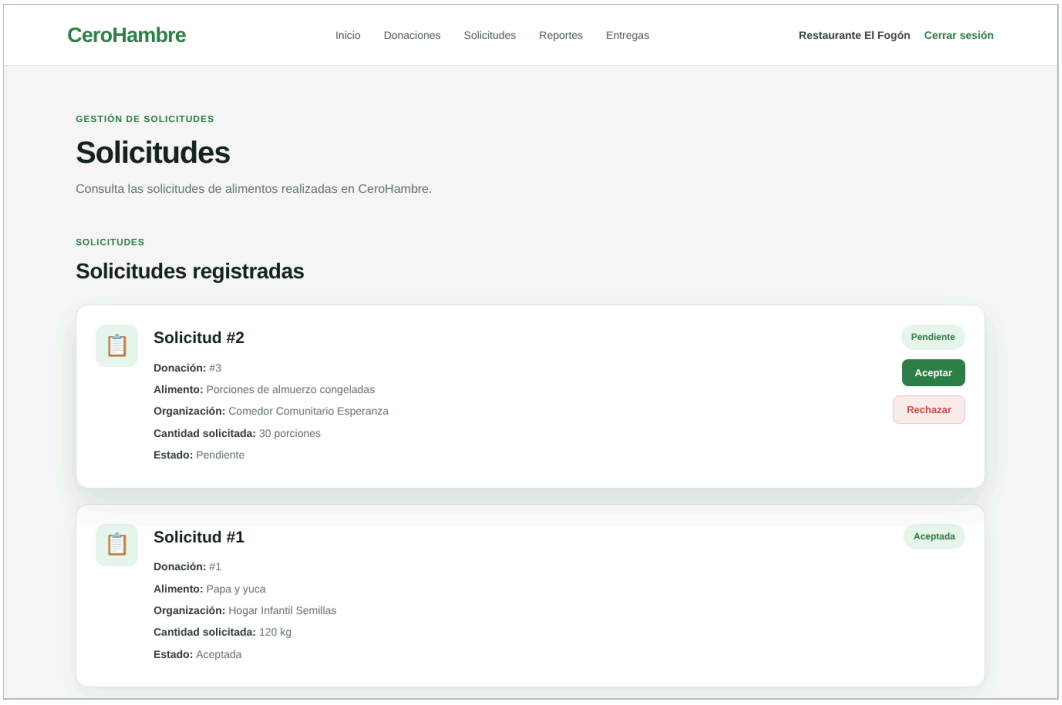



**Figura 7.** Solicitudes recibidas por el donante, con las acciones Aceptar y Rechazar.

## 4.7 Entregas

Una solicitud aceptada permite registrar la entrega. El donante la marca como en camino y, al confirmarse, como entregada; en ese momento la solicitud pasa a completada y la donación a entregada.

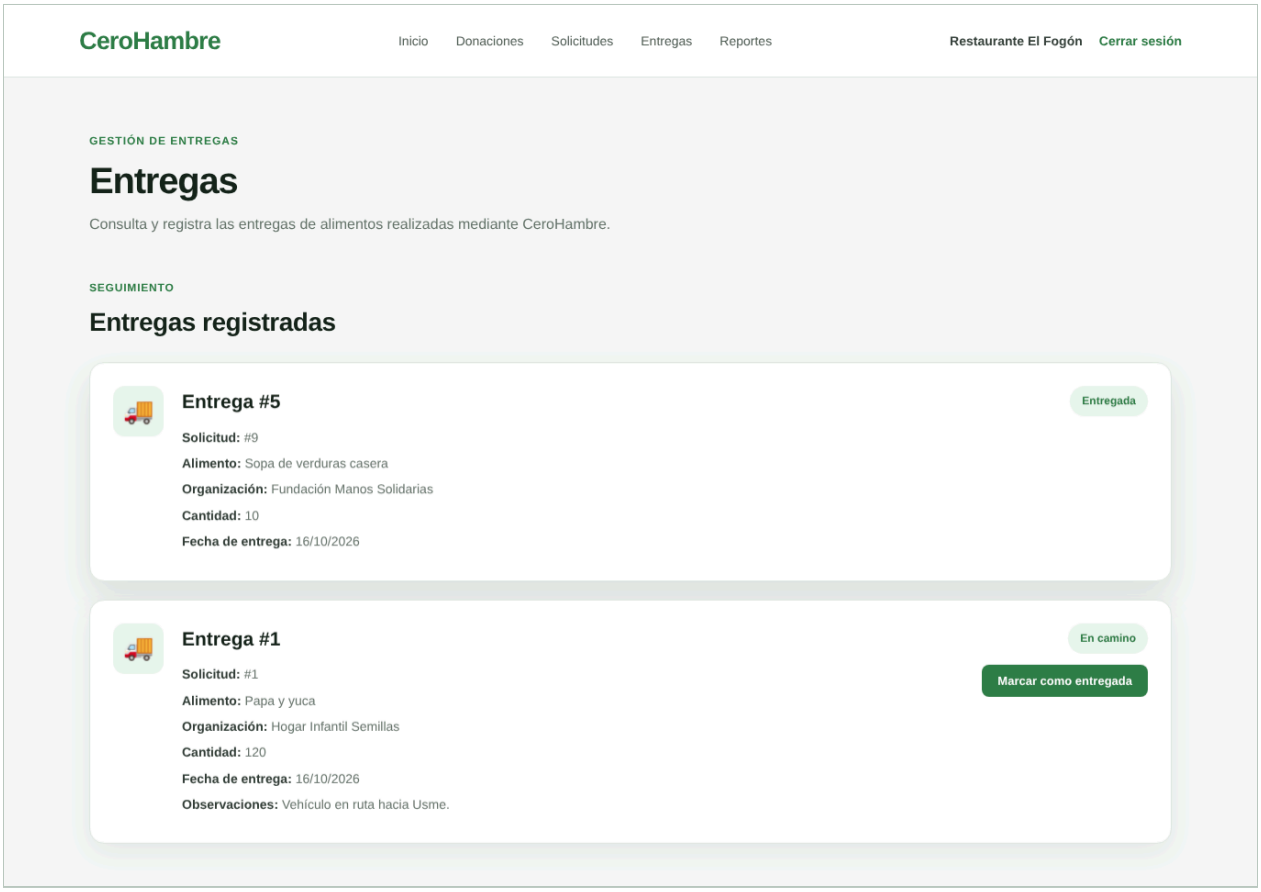



**Figura 8.** Seguimiento de entregas: una entregada y otra en camino.

## 4.8 Reportes

Los reportes calculan, a partir de la base de datos, los alimentos recuperados (kilogramos), las organizaciones atendidas, las entregas realizadas y las organizaciones beneficiadas, además del total recuperado por unidad de medida y el número de donaciones por estado.

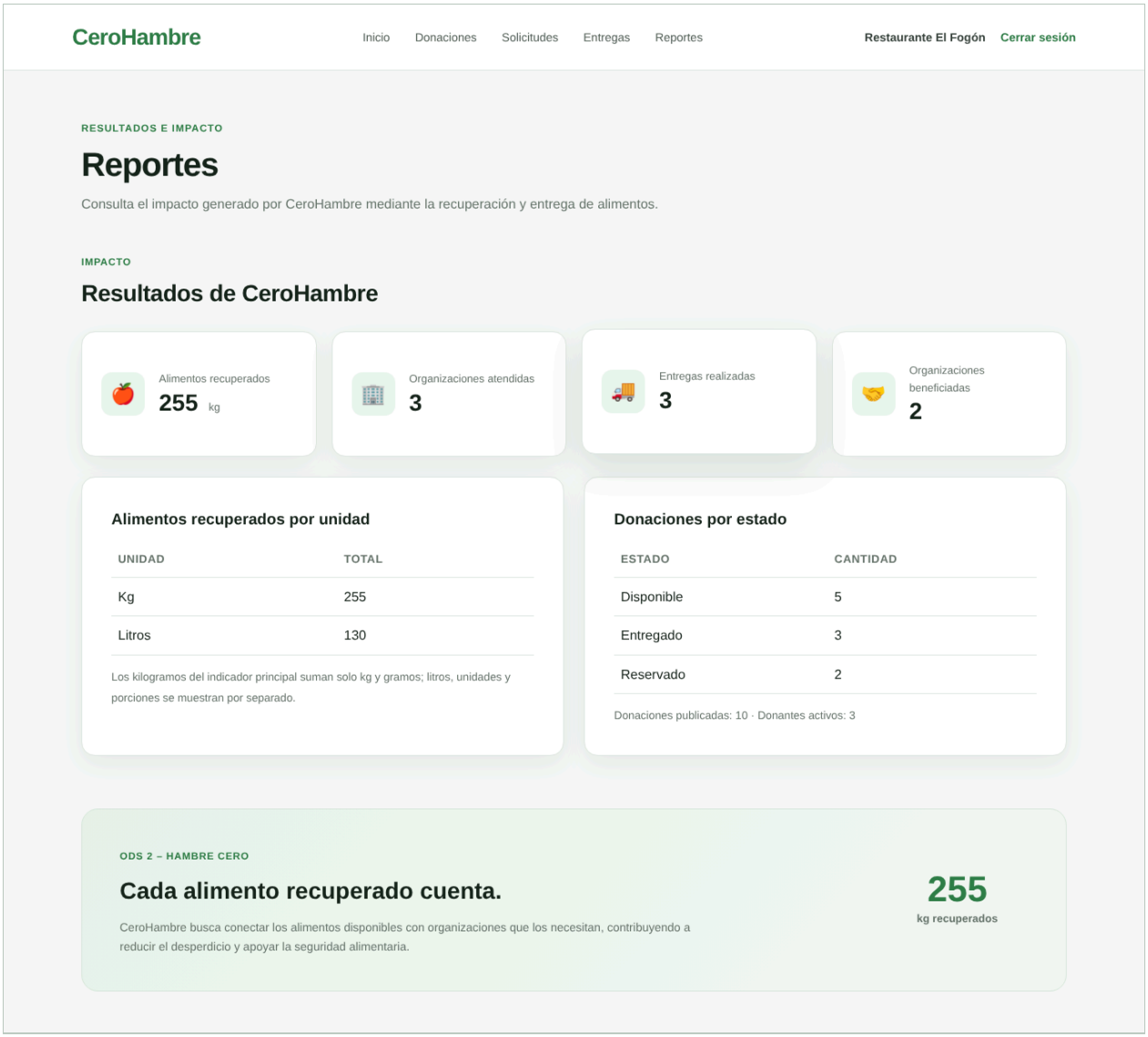



**Figura 9.** Reportes de impacto de CeroHambre.

# 5. Base de datos

## 5.1 Motor de base de datos

Se utiliza **PostgreSQL**, un motor relacional de código abierto con soporte completo de transacciones, llaves foráneas y restricciones de integridad. En producción se emplea el servicio PostgreSQL administrado de Render, al que la aplicación se conecta mediante la variable de entorno `DATABASE_URL`; en desarrollo y pruebas se usó PostgreSQL 16 local. La base es propia y persistente (no depende de servicios como Supabase o Neon).

## 5.2 Modelo relacional

| Tabla | Propósito | Llave primaria | Llaves foráneas |
|---|---|---|---|
| usuarios | Donantes y organizaciones registradas | id | — |
| donaciones | Alimentos que publica un donante | id | usuario_id → usuarios |
| solicitudes | Peticiones de una organización sobre una donación | id | donacion_id → donaciones<br>organizacion_id → usuarios |
| entregas | Seguimiento logístico de una solicitud aceptada | id | solicitud_id → solicitudes (único) |

### 5.3 Fragmento del esquema

Fragmento del archivo `database/schema.sql`: tabla `usuarios`, tabla `solicitudes` y el índice único parcial que impide solicitudes activas duplicadas.

```
CREATE TABLE IF NOT EXISTS usuarios (
    id              INTEGER GENERATED ALWAYS AS IDENTITY,
    nombre          VARCHAR(100) NOT NULL,
    correo          VARCHAR(150) NOT NULL,
    password_hash   VARCHAR(255) NOT NULL,           -- scrypt, nunca texto plano
    tipo_usuario    VARCHAR(20)  NOT NULL,
    ubicacion       VARCHAR(150) NOT NULL,
    fecha_registro  TIMESTAMPTZ  NOT NULL DEFAULT NOW(),

    CONSTRAINT usuarios_pk          PRIMARY KEY (id),
    CONSTRAINT usuarios_correo_uk   UNIQUE (correo),
    CONSTRAINT usuarios_correo_ck   CHECK (correo = LOWER(correo)),
    CONSTRAINT usuarios_tipo_ck     CHECK (tipo_usuario IN ('donante', 'organizacion'))
);
```

```
CREATE TABLE IF NOT EXISTS solicitudes (
    id                    INTEGER GENERATED ALWAYS AS IDENTITY,
    donacion_id           INTEGER        NOT NULL,
    organizacion_id       INTEGER        NOT NULL,     -- usuario tipo 'organizacion'
    cantidad_solicitada   NUMERIC(10, 2) NOT NULL,
    estado                VARCHAR(20)    NOT NULL DEFAULT 'pendiente',
    fecha_solicitud       TIMESTAMPTZ    NOT NULL DEFAULT NOW(),

    CONSTRAINT solicitudes_pk              PRIMARY KEY (id),
    CONSTRAINT solicitudes_donacion_fk     FOREIGN KEY (donacion_id)
        REFERENCES donaciones (id) ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT solicitudes_organizacion_fk FOREIGN KEY (organizacion_id)
        REFERENCES usuarios (id) ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT solicitudes_cantidad_ck     CHECK (cantidad_solicitada > 0),
    CONSTRAINT solicitudes_estado_ck       CHECK (estado IN ('pendiente', 'aceptada',
                                          'rechazada', 'completada'))
);

CREATE UNIQUE INDEX IF NOT EXISTS solicitudes_activa_uk
    ON solicitudes (donacion_id, organizacion_id)
    WHERE estado IN ('pendiente', 'aceptada');
```

## 5.4 Diagrama entidad-relación

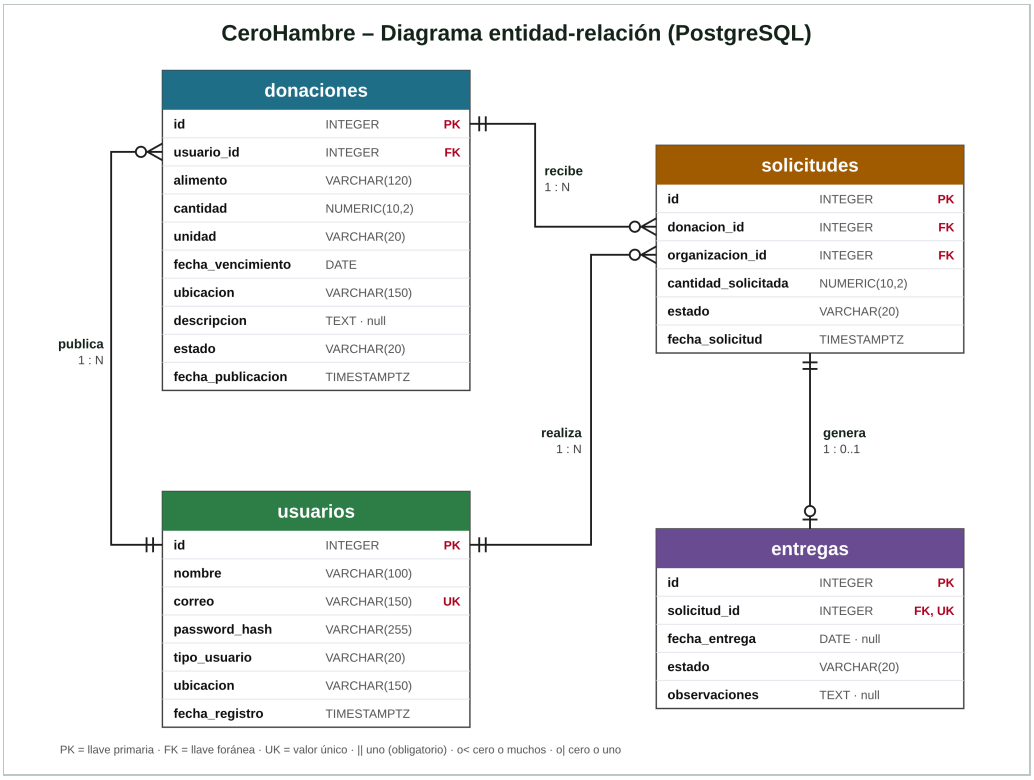



**Figura 10.** Diagrama entidad-relación de CeroHambre, generado a partir de la base de datos real (`database/DER.png`, alta resolución).

## 5.5 Cardinalidades

| Relación | Cardinalidad | Descripción |
|---|---|---|
| usuarios (donante) – donaciones | 1 : N | Un donante publica muchas donaciones; cada donación pertenece a un donante. |
| donaciones – solicitudes | 1 : N | Una donación puede recibir varias solicitudes; cada solicitud es sobre una donación. |
| usuarios (organización) – solicitudes | 1 : N | Una organización realiza muchas solicitudes. |
| solicitudes – entregas | 1 : 0..1 | Una solicitud aceptada genera como máximo una entrega. |

## 5.6 Normalización (3FN)

El modelo cumple la **Tercera Forma Normal**: (1) todos los atributos son atómicos y cada tabla tiene una llave primaria simple; (2) no existen dependencias parciales,

porque las llaves son de una sola columna; y (3) no existen dependencias transitivas, porque los datos de cada entidad se guardan una sola vez y se referencian por llave foránea: los datos del donante y de la organización viven solo en `usuarios`, los del alimento en `donaciones`, la cantidad pedida en `solicitudes` y el seguimiento en `entregas`. Valores derivados, como el hecho de que una donación esté vencida, no se almacenan: se calculan al consultar.

### 5.7 Integridad de los datos

- **Llaves foráneas con `ON DELETE RESTRICT`:** no se puede borrar un registro que tiene información relacionada.
- **Restricciones `CHECK`:** cantidades mayores que cero; unidades, estados y tipos de usuario dentro de listas cerradas; correo en minúsculas.
- **Restricciones `UNIQUE`:** un correo por usuario y una entrega por solicitud.
- **Índice único parcial:** una organización no puede tener dos solicitudes activas (pendiente o aceptada) sobre la misma donación.
- **Transacciones:** las operaciones de varios pasos (aceptar una solicitud, registrar o cerrar una entrega) se ejecutan en una transacción, con bloqueo de fila para evitar condiciones de carrera.

### 5.8 Archivos entregados en /database

| Archivo | Contenido |
|---|---|
| `schema.sql` | Esquema completo: tablas, llaves, restricciones e índices. |
| `seed.sql` | Datos de ejemplo (6 usuarios, 9 donaciones, 8 solicitudes y 4 entregas). |
| `cerohambre_backup.sql` | Volcado (*dump*) de la base con esquema y datos, listo para restaurar con `psql`. |
| `DER.png` / `DER.svg` | Diagrama entidad-relación en alta resolución. |

# 6. Tecnologías y justificación de los cambios

## 6.1 Tecnologías utilizadas

| Capa | Tecnología | Justificación |
|---|---|---|
| Frontend | HTML5, CSS3 y JavaScript | Sin pasos de compilación ni dependencias: el servidor entrega los archivos tal cual y la interfaz es fácil de mantener. |
| Backend | Node.js y Express 5 | Mismo lenguaje en cliente y servidor, ecosistema amplio y despliegue sencillo en Render. |
| Base de datos | PostgreSQL y el controlador `pg` | Motor relacional robusto, con integridad referencial, transacciones y servicio administrado en Render. Consultas parametrizadas y *pool* de conexiones. |
| Contraseñas | Hash `scrypt` (módulo `crypto` de Node) | Las contraseñas no se guardan en texto plano; `scrypt` es lento por diseño y resiste ataques de fuerza bruta. |
| Sesiones | Token firmado (JWT, HS256) | Permite autenticar cada petición sin guardar sesiones en el servidor y aplicar permisos por rol. |
| Despliegue | Render (servicio web + PostgreSQL) y `render.yaml` | Plataforma exigida por la actividad; el archivo Blueprint crea ambos servicios ya conectados. |
| Control de versiones | Git y GitHub | Historial de cambios y despliegue automático a partir del repositorio. |

## 6.2 Cambios respecto al Momento 1

Para pasar del prototipo inicial a una aplicación completa, segura y desplegable se realizaron los siguientes cambios, cada uno con su justificación:

| Cambio | Justificación |
|---|---|
| Base de datos: de MySQL a PostgreSQL | Render ofrece PostgreSQL administrado (no MySQL) y la actividad exige un motor propio y persistente. El esquema se rediseñó con llaves foráneas, restricciones e índices. |
| Configuración por variables de entorno (`.env` y `.env.example`) | Las credenciales no deben estar en el código ni en GitHub; en Render se configuran como variables del servicio. |

| Cambio | Justificación |
|---|---|
| Contraseñas con hash `scrypt` | Antes se guardaban y comparaban en texto plano, lo que es inseguro. |
| Autenticación con token y permisos por rol en el servidor | Antes bastaba con enviar el identificador del usuario desde el navegador; ahora el servidor verifica quién es y qué puede hacer. |
| Validaciones y reglas de negocio en el servidor, con transacciones | Evita datos inválidos y estados inconsistentes (cantidades excedidas, solicitudes duplicadas, donaciones vencidas o ya reservadas). |
| Interfaz: dirección de la API relativa, escape de HTML, vistas protegidas, botones según el rol, fechas dd/mm/aaaa y ventanas modales con estilo | La aplicación funciona en cualquier dominio, evita inyección de código (XSS) y ofrece una experiencia coherente para cada rol. |
| Reportes ampliados e inicio con cifras reales | Las cifras de impacto se calculan desde la base de datos (por unidad y por estado), en lugar de valores fijos. |
| Carpeta `/database` con esquema, datos de ejemplo, volcado y diagrama ER | Requisito de la actividad y facilita reproducir la base en cualquier equipo. |
| Despliegue en Render con `render.yaml`; el servidor Express también entrega el frontend | Un único servicio publica toda la aplicación y se actualiza con cada *push* al repositorio. |

### 6.3 Despliegue en Render

El repositorio incluye `render.yaml`: al crear un *Blueprint* en Render apuntando al repositorio, se crean la base de datos PostgreSQL y el servicio web, ya conectados. Al arrancar, la aplicación crea las tablas y carga los datos de ejemplo si la base está vacía. Variables de entorno:

- `DATABASE_URL`: conexión a PostgreSQL (la inyecta Render).
- `JWT_SECRET`: llave para firmar las sesiones (Render genera un valor aleatorio).
- `NODE_ENV`: `production` en el servidor desplegado.
- Opcionales: `ZONA_HORARIA`, `SESION_HORAS` y `CARGAR_DATOS_EJEMPLO`.

# 7. Validaciones, seguridad y pruebas

## 7.1 Validaciones y manejo de errores

| Capa | Qué se valida |
|---|---|
| Navegador | Campos obligatorios, formato de correo, contraseña de mínimo 8 caracteres, cantidades positivas y fechas. |
| Servidor | Tipos y longitudes de los datos, unidades y estados permitidos, fechas válidas y no vencidas, identificadores numéricos y propiedad del recurso. |
| Negocio | Cantidad solicitada menor o igual a la disponible; donación no vencida ni reservada; sin solicitudes activas duplicadas; transiciones de estado válidas. |
| Base de datos | Restricciones `NOT NULL`, `CHECK`, `UNIQUE` y llaves foráneas. |

Los errores se devuelven como JSON con el código HTTP adecuado (400, 401, 403, 404 o 409) y la interfaz los muestra con un mensaje claro; un manejador central impide que un dato incorrecto detenga el servidor.

## 7.2 Seguridad

- Contraseñas con hash `scrypt` y comparación en tiempo constante.
- Consultas SQL parametrizadas (sin concatenar datos del usuario), que previenen la inyección SQL.
- Escape del texto de los usuarios antes de mostrarlo (prevención de XSS).
- Permisos por rol verificados en el servidor y sesiones con vencimiento.
- El archivo `.env` está excluido del repositorio; solo se publica `.env.example`.

## 7.3 Pruebas realizadas

Se ejecutaron pruebas automáticas de la API sobre una base PostgreSQL real (`npm test`, con `DATABASE_URL_TEST`), que verifican el registro e inicio de sesión, los permisos por rol, la validación de datos, la regla de cantidad máxima, la aceptación de solicitudes y su efecto sobre la donación, el ciclo de vida de las entregas y los reportes.

También se probó en un navegador el flujo completo desde la interfaz: acceso sin sesión, clave incorrecta, publicación, fecha de vencimiento pasada, texto con código HTML, cantidad superior a la disponible, solicitud, aceptación, entrega, reportes y registro de un usuario nuevo.

## 8. Conclusiones

CeroHambre ofrece un flujo completo y verificable para que los alimentos excedentes lleguen a organizaciones que los necesitan, con reglas que evitan errores operativos, permisos por rol, una base de datos relacional normalizada y un despliegue reproducible en Render. De esta forma, la solución aporta de manera concreta al ODS 2 – Hambre Cero y a la reducción del desperdicio de alimentos.

Como trabajo futuro se proponen las notificaciones por correo, la búsqueda de donaciones cercanas mediante geolocalización, un panel de administración y el registro del número de personas atendidas por cada organización para medir el impacto con mayor precisión.